TP_modul_6

Nama : Helven Marcelyno Aldy Dwy Anggara

Nim : 108092500012

Kelas : IT-08-02

FILE CODE

1. DLL.h

DLL.h | DLL.cpp | main.cpp

```
#ifndef DLL_H_INCLUDED
#define DLL_H_INCLUDED

#include <iostream>
#include <string>

using namespace std;

struct infotype {
    string namaBand;
    string judulLagu;
};

typedef struct elmtList *address;

struct elmtList {
    infotype info;
    address prev;
    address next;
};

struct List {
    address first;
    address last;
};

bool isEmpty(List L);
void createList(List &L);
address createNewElmt(infotype X);
void insertFirst(List &L, address P);
void insertAfter(List &L, address Prec, address P);
void insertLast(List &L, address P);
void deleteFirst(List &L, address &P);
void deleteAfter(List &L, address Prec, address &P);
void deleteLast(List &L, address &P);
void concat(List L1, List L2, List &L3);
address findLagu(string Judul, List L);
void removeLagu(string Judul, List &L);
void printList(List L);

#endif

```

2. DLL.cpp

DLL.h | DLL.cpp | main.cpp

```cpp
#include "DLL.h"

bool isEmpty(List L) {
    return L.first == NULL && L.last == NULL;
}

void createList(List &L) {
    L.first = NULL;
    L.last = NULL;
}

address createNewElmt(infotype X) {
    address P = new elmtList;
    if (P != NULL) {
        P->info = X;
        P->prev = NULL;
        P->next = NULL;
    }
    return P;
}

void insertFirst(List &L, address P) {
    if (isEmpty(L)) {
        L.first = P;
        L.last = P;
    } else {
        P->next = L.first;
        L.first->prev = P;
        L.first = P;
    }
}

void insertAfter(List &L, address Prec, address P) {
    if (Prec != NULL) {
        P->next = Prec->next;
        P->prev = Prec;
        if (Prec->next != NULL) {
            Prec->next->prev = P;
        } else {
            L.last = P;
        }
        Prec->next = P;
    }
}
```

DLL.h DLL.cpp main.cpp

```
84  }
85
86  void deleteLast(List &L, address &P) {
87      if (!isEmpty(L)) {
88          P = L.last;
89          if (L.first == L.last) {
90              L.first = NULL;
91              L.last = NULL;
92          } else {
93              L.last = L.last->prev;
94              L.last->next = NULL;
95          }
96          P->next = NULL;
97          P->prev = NULL;
98      }
99  }
100
101 void concat(List L1, List L2, List &L3) {
102     createList(L3);
103     address P = L1.first;
104     while (P != NULL) {
105         address baru = createNewElmt(P->info);
106         insertLast(L3, baru);
107         P = P->next;
108     }
109     P = L2.first;
110     while (P != NULL) {
111         address baru = createNewElmt(P->info);
112         insertLast(L3, baru);
113         P = P->next;
114     }
115 }
116
117 address findLagu(string Judul, List L) {
118     address P = L.first;
119     while (P != NULL) {
120         if (P->info.judulLagu == Judul) {
121             return P;
122         }
123         P = P->next;
124     }
125     return NULL;
126 }
```

DLL.h | DLL.cpp | main.cpp

```
43      }
44  }
45
46  void insertLast(List &L, address P) {
47      if (isEmpty(L)) {
48          insertFirst(L, P);
49      } else {
50          P->prev = L.last;
51          L.last->next = P;
52          L.last = P;
53      }
54  }
55
56  void deleteFirst(List &L, address &P) {
57      if (!isEmpty(L)) {
58          P = L.first;
59          if (L.first == L.last) {
60              L.first = NULL;
61              L.last = NULL;
62          } else {
63              L.first = L.first->next;
64              L.first->prev = NULL;
65          }
66          P->next = NULL;
67          P->prev = NULL;
68      }
69  }
70
71  void deleteAfter(List &L, address Prec, address &P) {
72      if (Prec != NULL && Prec->next != NULL) {
73          P = Prec->next;
74          if (P == L.last) {
75              L.last = Prec;
76              Prec->next = NULL;
77          } else {
78              Prec->next = P->next;
79              P->next->prev = Prec;
80          }
81          P->next = NULL;
82          P->prev = NULL;
83      }
84  }
```

```
127
128 void removeLagu(string Judul, List &L) {
129     address P = findLagu(Judul, L);
130     if (P != NULL) {
131         address temp;
132         if (P == L.first) {
133             deleteFirst(L, temp);
134         } else if (P == L.last) {
135             deleteLast(L, temp);
136         } else {
137             address Prec = P->prev;
138             deleteAfter(L, Prec, temp);
139         }
140         delete temp;
141     }
142 }
143
144 void printList(List L) {
145     address P = L.first;
146     if (isEmpty(L)) {
147         cout << "List Kosong\n";
148     } else {
149         while (P != NULL) {
150             cout << "Band: " << P->info.namaBand << " | Lagu: " << P->info.jud
151             P = P->next;
152         }
153     }
154     cout << "\n";
155 }
156
```

Main.cpp

DLL.h | DLL.cpp | main.cpp

```
    List L1, L2, L3;
    infotype data;
    address P;

    createList(L1);
    createList(L2);

    cout << "=== MENGISI LIST 1 ===\n";
    data.namaBand = "Foster The People";
    data.judulLagu = "Imagination";
    P = createNewElmt(data);
    insertFirst(L1, P);

    data.namaBand = "The Corrs";
    data.judulLagu = "Radio";
    P = createNewElmt(data);
    insertLast(L1, P);

    cout << "Isi List 1:\n";
    printList(L1);

    cout << "=== MENGISI LIST 2 ===\n";
    data.namaBand = "Maroon 5";
    data.judulLagu = "Sugar";
    P = createNewElmt(data);
    insertFirst(L2, P);

    data.namaBand = "Coldplay";
    data.judulLagu = "Yellow";
    P = createNewElmt(data);
    insertLast(L2, P);

    cout << "Isi List 2:\n";
    printList(L2);

    cout << "=== CONCAT LIST 1 & LIST 2 KE LIST 3 ===\n";
    concat(L1, L2, L3);
    cout << "Isi List 3 (Hasil Gabungan):\n";
    printList(L3);

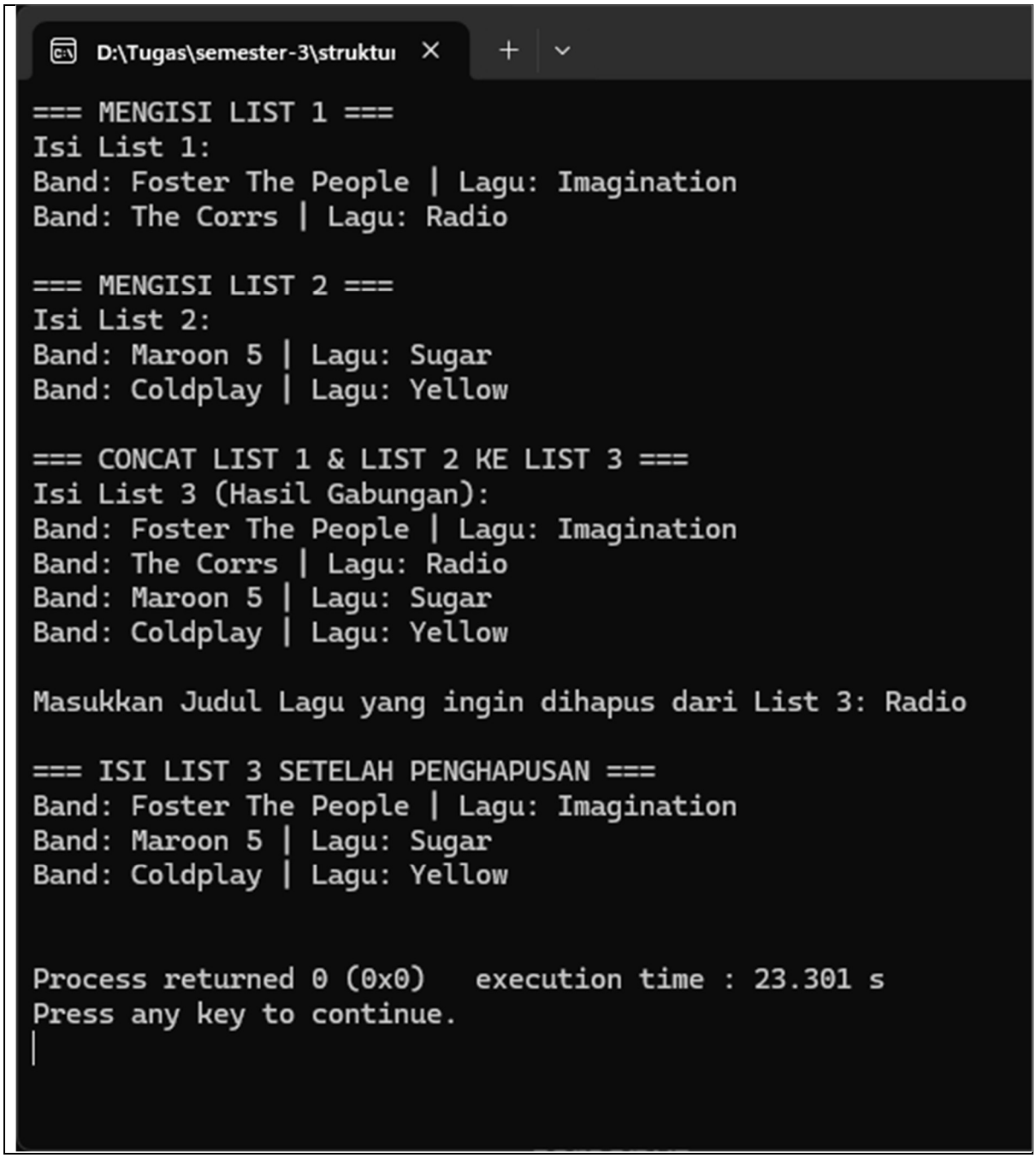
    string laguDihapus;
    cout << "Masukkan Judul Lagu yang ingin dihapus dari List 3: ";
    getline(cin, laguDihapus);

    removeLagu(laguDihapus, L3);

    cout << "\n=== ISI LIST 3 SETELAH PENGHAPUSAN ===\n";
    printList(L3);

    return 0;
}
```

BUILD & RUN


D:\Tugas\semester-3\struktur
=== MENGISI LIST 1 ===
Isi List 1:
Band: Foster The People | Lagu: Imagination
Band: The Corrs | Lagu: Radio
=== MENGISI LIST 2 ===
Isi List 2:
Band: Maroon 5 | Lagu: Sugar
Band: Coldplay | Lagu: Yellow
=== CONCAT LIST 1 & LIST 2 KE LIST 3 ===
Isi List 3 (Hasil Gabungan):
Band: Foster The People | Lagu: Imagination
Band: The Corrs | Lagu: Radio
Band: Maroon 5 | Lagu: Sugar
Band: Coldplay | Lagu: Yellow
Masukkan Judul Lagu yang ingin dihapus dari List 3: Radio
=== ISI LIST 3 SETELAH PENGHAPUSAN ===
Band: Foster The People | Lagu: Imagination
Band: Maroon 5 | Lagu: Sugar
Band: Coldplay | Lagu: Yellow
Process returned 0 (0x0) execution time : 23.301 s
Press any key to continue.